ANGULAR
ARCHITECTS

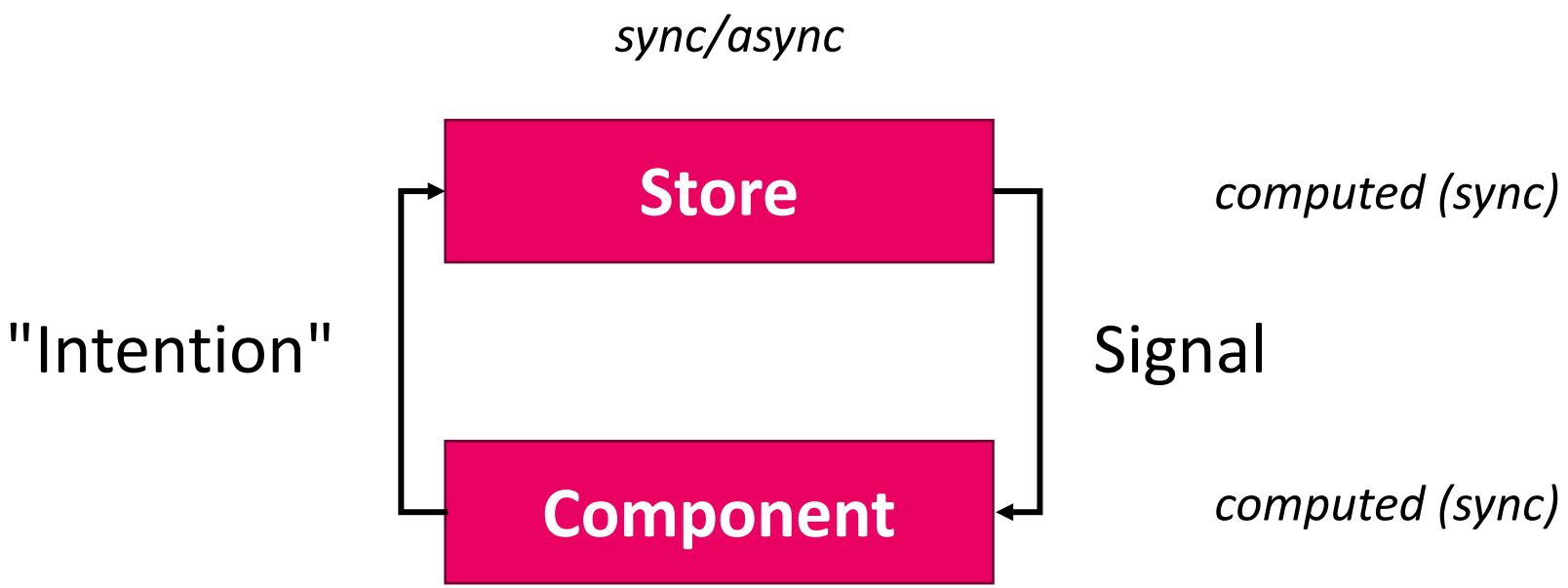
NGRX Signal Store

ANGULARarchitects.io

@Manfred Steyer

Stores Streamline Reactive Flow

ANGULAR ARCHITECTS



ANGULAR ARCHITECTS

First Steps

Signal Store

ANGULAR ARCHITECTS

```
export const FlightBookingStore = signalStore(
  { providedIn: 'root' },
  […]
);
```

Signal Store

ANGULAR ARCHITECTS

```
export const FlightBookingStore = signalStore(
  { providedIn: 'root' },
  withState({
    from: 'Paris',
    to: 'London',
    […]
  }),
  […]
);
```

Signal Store

ANGULAR ARCHITECTS

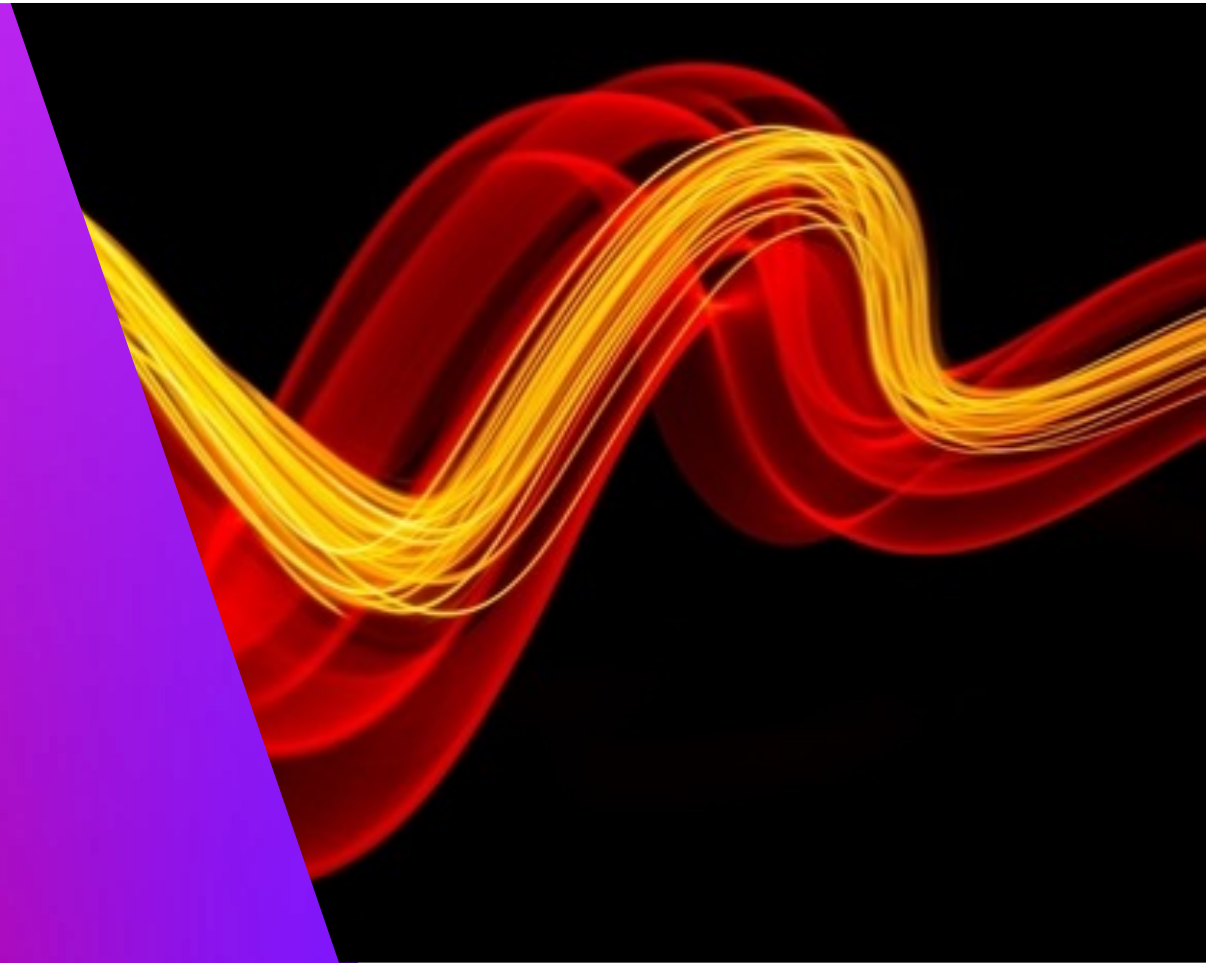
```
export const FlightBookingStore = signalStore(
  { providedIn: 'root' },
  withState({
    from: 'Paris',
    to: 'London',
    […]
  }),
  withComputed(([…]) => ({ […] })),
  withMethods(([…]) => ({ })),
  withHooks({ […] })
);
```

ANGULAR ARCHITECTS

DEMO

ANGULAR ARCHITECTS

RxJS Interop



rxMethod: Connecting Worlds

ANGULAR ARCHITECTS

```
export const FlightBookingStore = signalStore(
  […]
  withMethods((state) => {
    […]
    return {
      connectCriteria: rxMethod<Criteria>((c$) => c$.pipe([…]))
    };
  }),
  […]
);
```

rxMethod: Connecting Worlds

ANGULAR ARCHITECTS

```
export const FlightBookingStore = signalStore(
  […]
  withMethods((state) => {
    […]
    return {
      connectCriteria: rxMethod<Criteria>((c$) => c$.pipe([…]))
    };
  }),
  […]
);
```

rxMethod: Connecting Worlds

ANGULAR ARCHITECTS

```
export const FlightBookingStore = signalStore(
  […]
  withMethods((state) => {
    […]
    return {
      connectCriteria: rxMethod<Criteria>((c$) => c$.pipe(
        filter(c => c.from.length >= 3 && c.to.length >= 3),
        debounceTime(300),
        switchMap((c) => flightService.find(c.from, c.to)),
        tap(flights => patchState(state, { flights }))
      ))
    };
  }),
  […]
);
```

Calling an rxMethod

```
store.connectCriteria(criteria);
```

takes: Signal<T>, Observable<T>, T

ANGULAR ARCHITECTS

DEMO

ANGULAR ARCHITECTS

Entities

withEntities

```
const BookingStore = signalStore(
  withEntities<FlightState>()
);
```

withEntities

ANGULAR ARCHITECTS

State

entityMap: {
  17: { id: 17, … },
  18: { id: 18, … },
  19: { id: 19, … }
}

ids: [17, 18, 19]

```
patchState(
  state,
  setAllEntities(flights)
);
```

Computed

entities: [
  { id: 17, … },
  { id: 18, … },
  { id: 19, … }
]

Updaters

- addEntity
- addEntities
- removeEntity
- removeEntities
- removeAllEntities
- setEntity
- setEntities
- setAllEntities
- updateEntity
- updateEntities
- updateAllEntities

withEntities

ANGULAR ARCHITECTS

State

entityMap: {
  17: { id: 17, … },
  18: { id: 18, … },
  19: { id: 19, … }
}

ids: [17, 18, 19]

Computed

entities: [
  { id: 17, … },
  { id: 18, … },
  { id: 19, … }
]

Updaters

- addEntity
- addEntities
- removeEntity
- removeEntities
- removeAllEntities
- setEntity
- setEntities
- setAllEntities
- updateEntity
- updateEntities
- updateAllEntities

```
patchState(
  state,
  setAllEntities(flights, { idKey: 'flightId' })
);
```

withEntities

```
const BookingStore = signalStore(
  withEntities({
    entity: type<FlightState>(),
    collection: 'flight'
  }),
  withEntities({
    entity: type<PassengerState>(),
    collection: 'passenger'
  }),
);
```

withEntities

ANGULAR ARCHITECTS

State

flightEntityMap: {
  17: { id: 17, … },
  18: { id: 18, … },
  19: { id: 19, … }
}

flightIds: [17, 18, 19]

Computed

flightEntities: [
  { id: 17, … },
  { id: 18, … },
  { id: 19, … }
]

Upaters

- addEntity
- addEntities
- removeEntity
- removeEntities
- removeAllEntities
- setEntity
- setEntities
- setAllEntities
- updateEntity
- updateEntities
- updateAllEntities

```
patchState(
  state,
  setAllEntities(flights, { collection: 'flight' })
);
```

ANGULAR ARCHITECTS

DEMO

Avoid Deep State

ANGULAR ARCHITECTS

state

↓

flights

↓

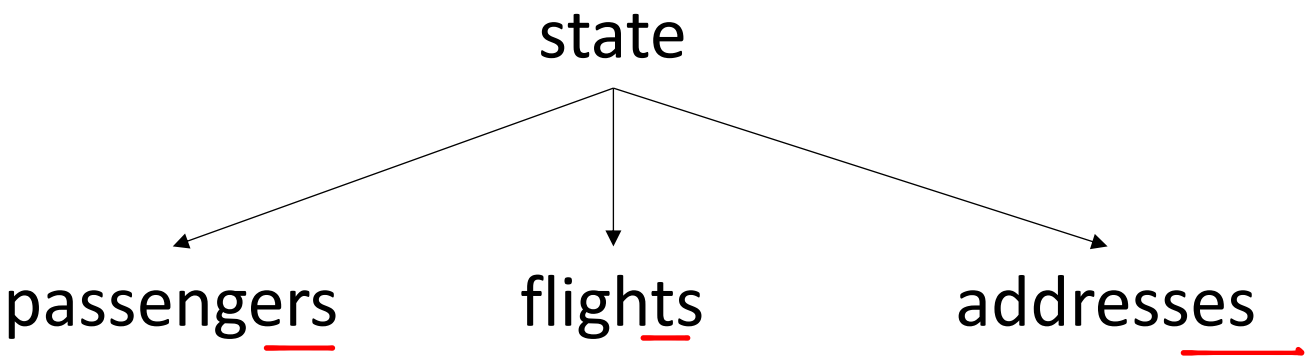
passengers

↓

addresses

Prefer Flat State

ANGULAR ARCHITECTS



Normalization

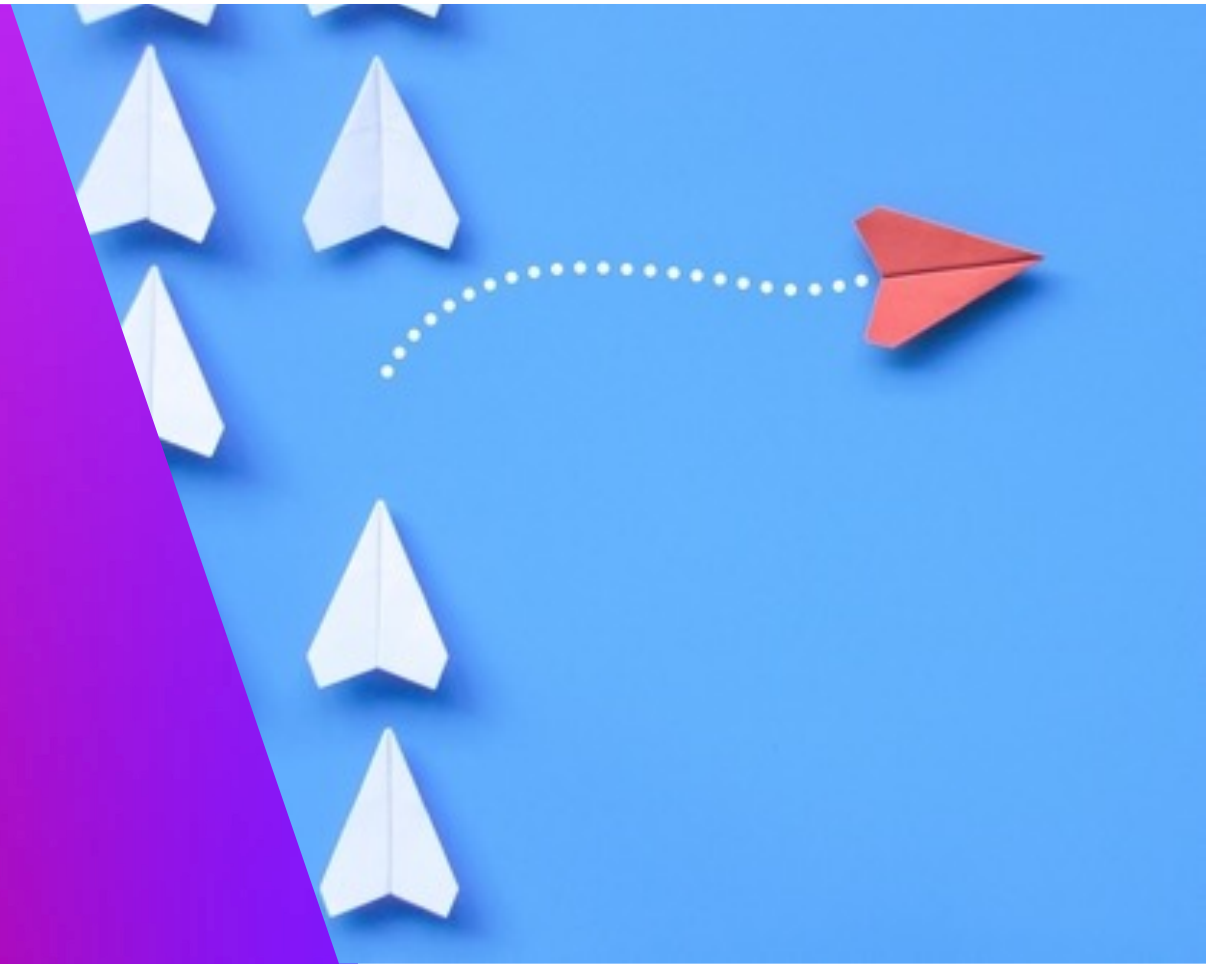
ANGULAR ARCHITECTS

```
[
  {
    id: 1,
    firstName: 'Max',
    lastName: 'Muster',
    flightIds: [10, 20, 30],
  },
  {
    id: 2,
    firstName: 'Susi',
    lastName: 'Sorglos',
    flightIds: [10, 20, 30],
  },
  {…}
]
```

```
[
  {
    id: 10,
    from: 'GRZ',
    to: 'MUC',
    passengerIds: [1, 2, 3],
  },
  {
    id: 20,
    from: 'MUC',
    to: 'GRZ',
    passengerIds: [1, 2, 3],
  },
  {…}
]
```

ANGULAR ARCHITECTS

Custom Features



Custom Features

ANGULAR ARCHITECTS

```
export const FlightBookingStore = signalStore(
  { providedIn: 'root' },
  withState({
    from: 'Paris',
    to: 'London',
    […]
  }),
  withSignals(([…]) => ({ […] })),
  withMethods(([…]) => ({ })),
  withHooks({ […] }),
  withCallState()
);
```

Custom Features

ANGULAR ARCHITECTS

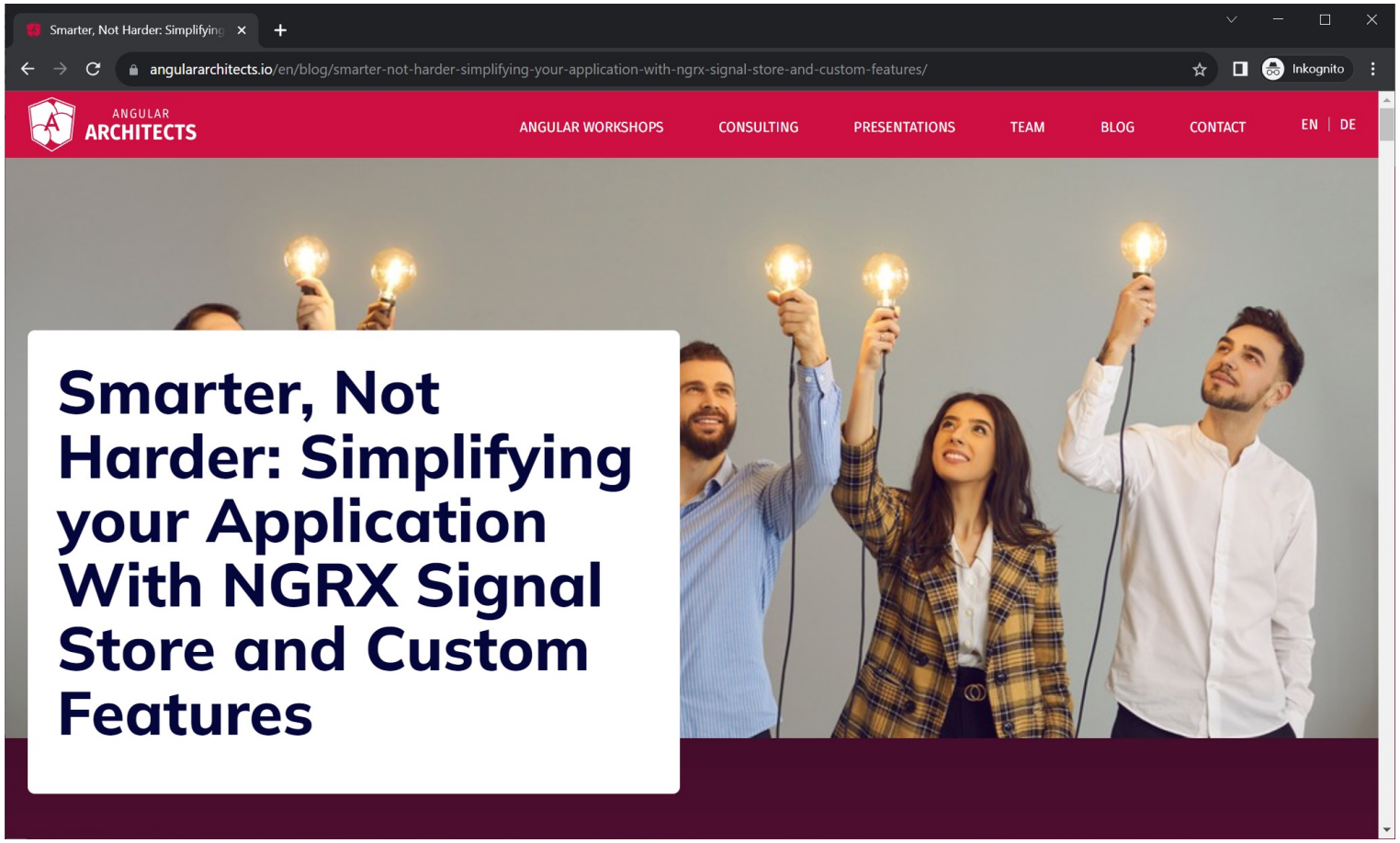
```
export const FlightBookingStore = signalStore(
  { providedIn: 'root' },
  withState({
    from: 'Paris',
    to: 'London',
    […]
  }),
  withSignals(([…]) => ({ […] })),
  withMethods(([…]) => ({ })),
  withHooks({ […] }),
  withCallState()
);
```

ANGULAR ARCHITECTS

DEMO

More on Custom Features

ANGULAR ARCHITECTS



Store for CRUD & Undo/Redo in 7 LOC

ANGULAR ARCHITECTS

```
1  export const FlightBookingStore = signalStore(
2    { providedIn: 'root' },
3    withEntities<Flight>(),
4    withCallState(),
5    withDataService(FlightService, { from: 'Graz', to: 'Hamburg'} ),
6    withUndoRedo(),
7  );
```

Link to Article

ANGULAR ARCHITECTS

Labs

ANGULAR
ARCHITECTS

Custom Features in Redux Toolkit

@Manfred Steyer

@angular-architects/ngrx-toolkit

0.0.3 • Public • Published a month ago

ANGULAR
ARCHITECTS

Readme | Code Beta | 1 Dependency | 0 Dependents

NgRx Toolkit

CRUD in a Few LOC

ANGULAR ARCHITECTS

```
export const PassengerStore = signalStore(
  { providedIn: 'root' },
  withCallState(),
  withEntities<Passenger>(),
  withDataService({
    dataServiceType: PassengerService,
    filter: { firstName: '', name: 'Smith' },
  }),
);
```

Undo/ Redo

ANGULAR ARCHITECTS

```
export const PassengerStore = signalStore(
  { providedIn: 'root' },
  withCallState(),
  withEntities<Passenger>(),
  withDataService({
    dataServiceType: PassengerService,
    filter: { firstName: '', name: 'Smith' },
  }),
  withUndoRedo(),
);
```

Redux DevTools Support

ANGULAR ARCHITECTS

```
export const PassengerStore = signalStore(
  […]
  withDevtools('passenger')
);
```

DevTools - localhost:4200/flight-booking/passenger-search

Elemente | Speicherverbrauch | Konsole | Netzwerk | Quellcode | Leistung | Redux

Actions | Settings

Reset | Revert | Sweep | Commit | NgRx Signal Store

filter...

Store Update	6:46:43.31
Store Update	+00:00.00
Store Update	+00:04.13
Store Update	+00:00.16
Store Update	+00:01.07
Store Update	+00:00.00
Store Update	+00:00.38

State | Action | State | Diff | Trace | Test

Tree | Chart | Raw

```
▾ passenger (pin)
    passengerCallState (pin): "loaded"
  ▾ passengerEntityMap (pin)
    ▸ 3016 (pin): { id: 3016, name: "Smith", firstName: "Emma",
    ▸ 3040 (pin): { id: 3040, name: "Smith", firstName: "Mia", …
    ▸ 3064 (pin): { id: 3064, name: "Smith", firstName: "Hanna",
    ▸ 3088 (pin): { id: 3088, name: "Smith", firstName: "Emilia"
```

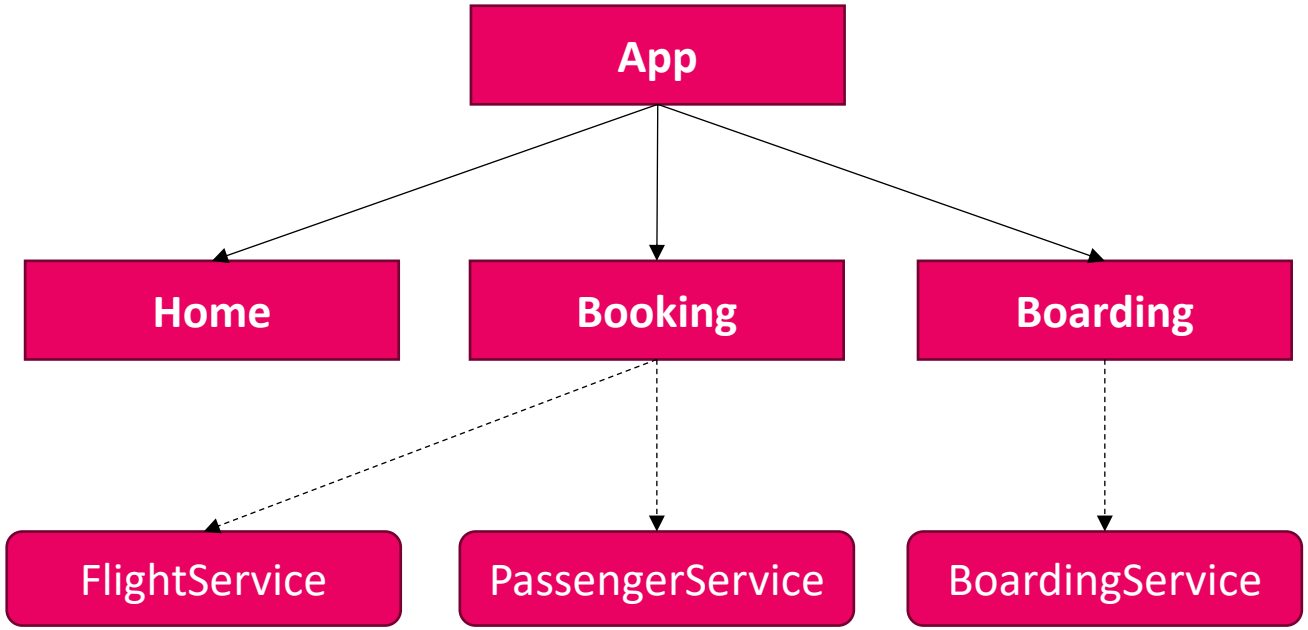
flights loaded (17)

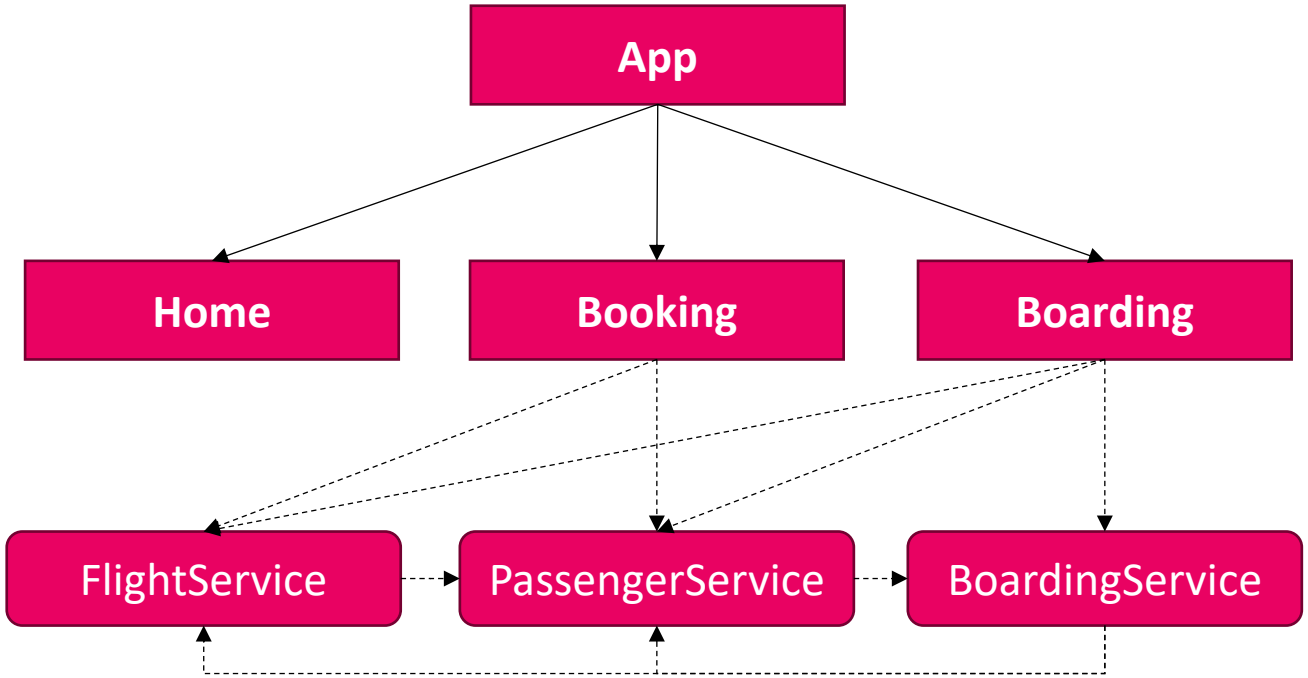
Live | 1x | 2x

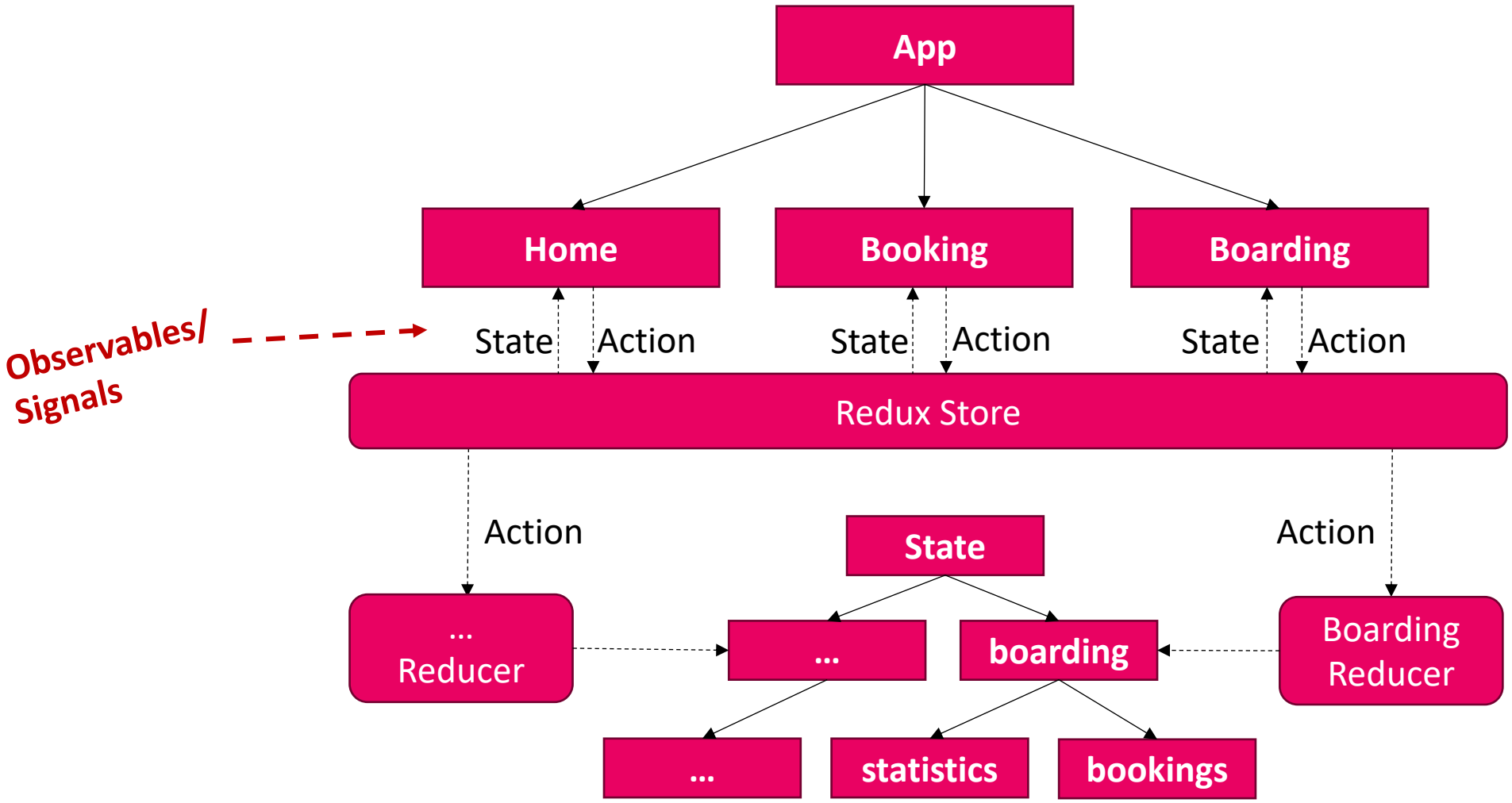
Inspector | Log monitor | Chart | RTK Query

ANGULAR ARCHITECTS

Redux Pattern

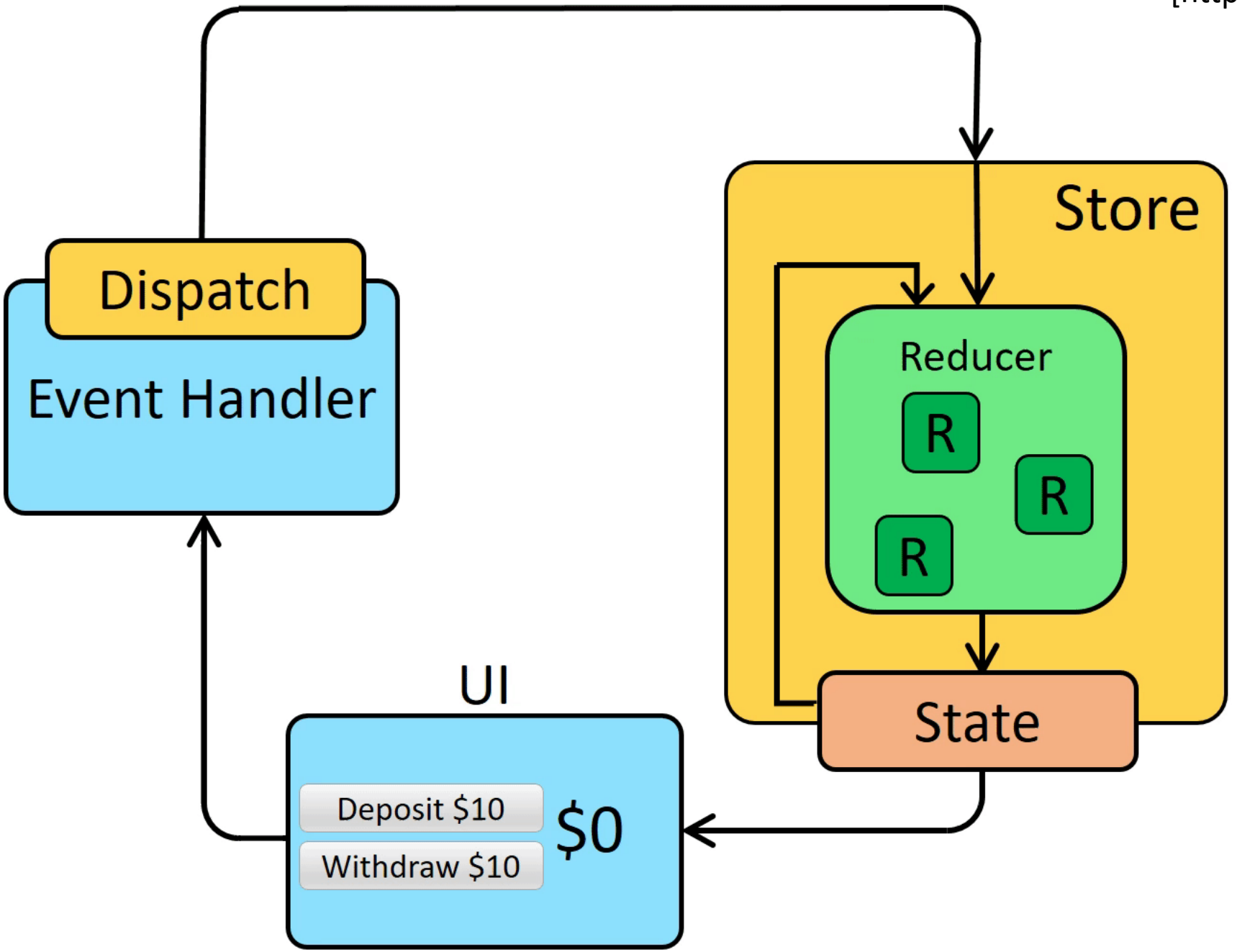

App
Home
Booking
Boarding
FlightService
PassengerService
BoardingService


App
Home
Booking
Boarding
FlightService
PassengerService
BoardingService

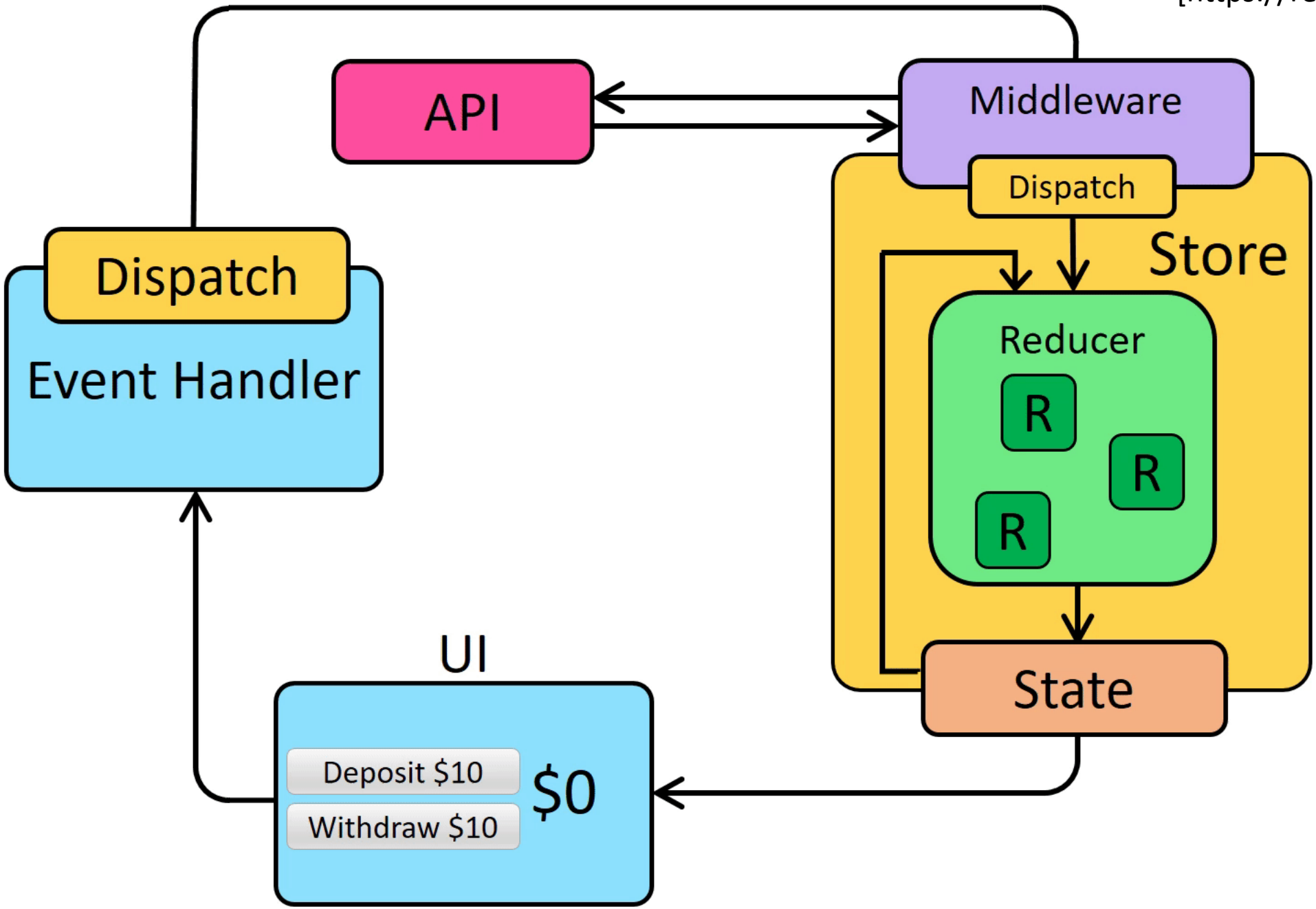


Single Immutable State Tree

[https://redux.js.org]



[https://redux.js.org]



Redux Support in NGRX Toolkit

ANGULAR ARCHITECTS

```
export const LuggageStore = signalStore(
  { providedIn: 'root' },
  withState({ luggage: [] as Luggage[] }),
  withRedux({
    […]
  }),
);
```

Actions

```
export const LuggageStore = signalStore(
  { providedIn: 'root' },
  withState({ luggage: [] as Luggage[] }),
  withRedux({
    […]
  }),
);
```

Reducer

ANGULAR ARCHITECTS

```
export const LuggageStore = signalStore(
  { providedIn: 'root' },
  withState({ luggage: [] as Luggage[] }),
  withRedux({
    […]
    reducer(actions, on) {
      on(actions.loaded, ({ luggage }, state) => {
        patchState(state, 'flights loaded', { luggage });
      });
    },
  }),
);
```

Effects

ANGULAR ARCHITECTS

```
export const LuggageStore = signalStore(
  { providedIn: 'root' },
  withState({ luggage: [] as Luggage[] }),
  withRedux({
    […]
    effects(actions, create) {
      const luggageService = inject(LuggageService);
      return {
        load$: create(actions.load).pipe(
          switchMap((a) => luggageService.load(a.passengerId)),
          tap((luggage) => actions.loaded({ luggage }))
        ),
      };
    },
}));
```

Consumer

ANGULAR ARCHITECTS

```
@Component({ […] })
export class CheckinComponent implements OnInit {
  luggageStore = inject(LuggageStore);

  // Reading State as Signal
  luggage = this.luggageStore.luggage;

  ngOnInit(): void {
    // Dispatching an Action
    this.luggageStore.load({ passengerId: 17 });
  }
}
```

ANGULAR ARCHITECTS

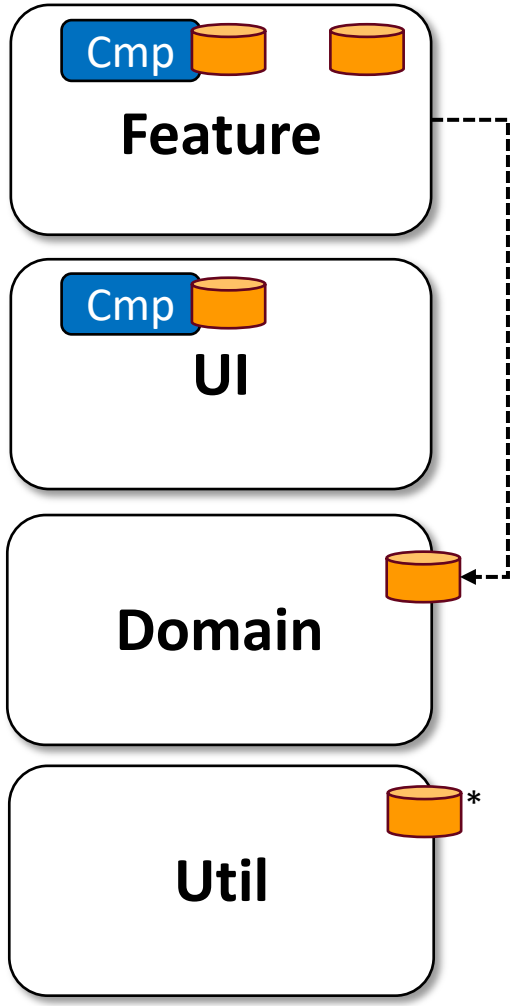
Labs

ANGULAR
ARCHITECTS

Signal Store and Your Architecture

@Manfred Steyer

Some Rules for Lightweight Stores



* seldom

Each module: Own state/ lightweight store

Component Level vs. forRoot

Hidden vs. Exposed

How Large should it be?

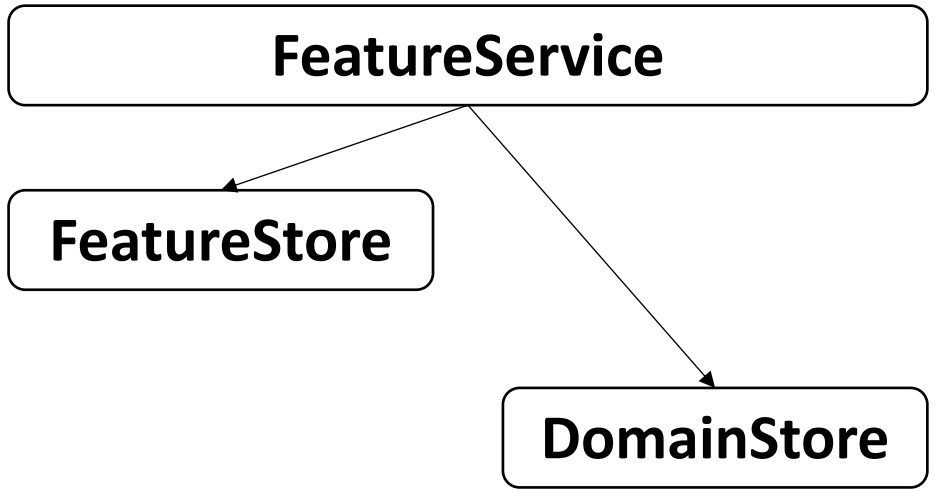
ANGULAR ARCHITECTS

Feature

Sub-Feature

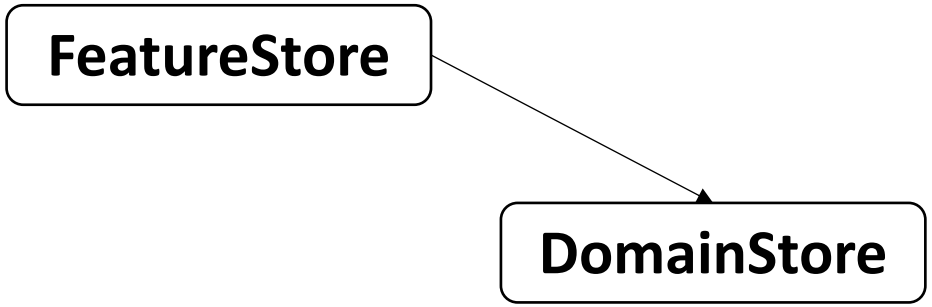
What if a Feature Needs Several Stores?

ANGULAR ARCHITECTS



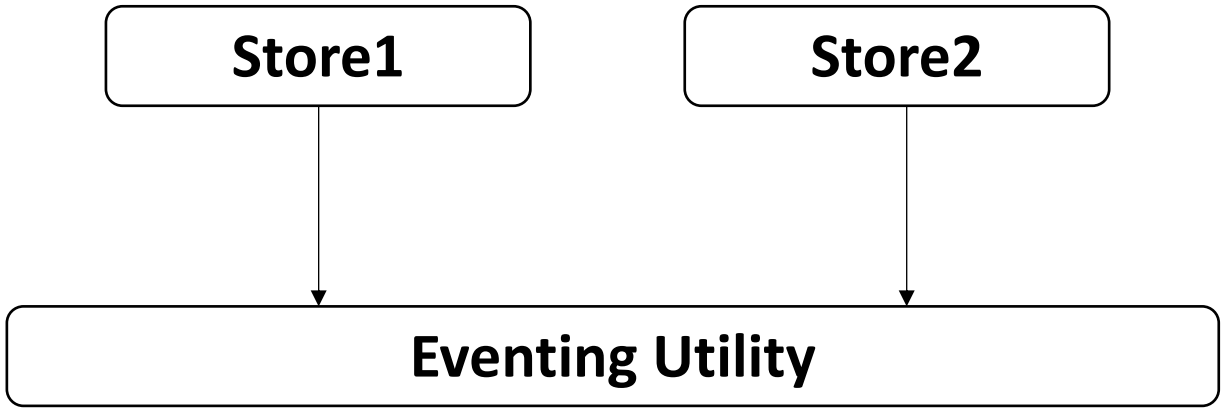
What if a Feature Needs Several Stores?

ANGULAR ARCHITECTS



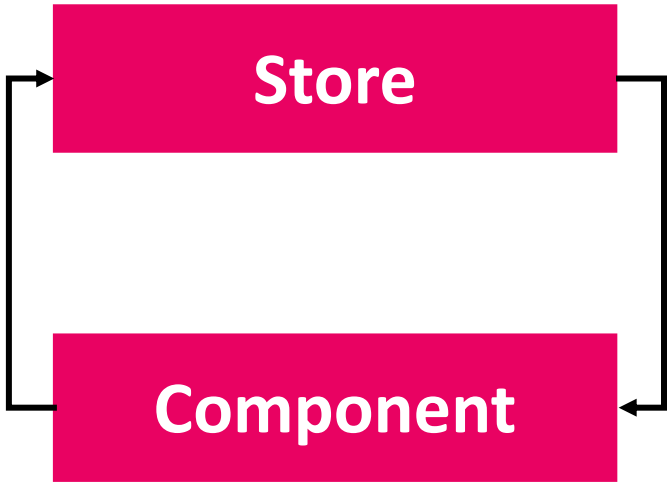
What if a Feature Needs Several Stores?

ANGULAR ARCHITECTS



Uni-Directional Data Flow

- Send "Intention" to Store
- Store performs sync or async operation and updates State
- Synchronously **derive state** (View Models) where possible (computed)
- Signals make Angular to update View



Smart vs. Dumb Components

ANGULAR ARCHITECTS

Smart Component	Dumb
• Drives the "Use Case" • Usually a "Container"	• Independent of Use Case • No access to feature or domain store • Reusable • Usually a "Leaf"

Summary: Patterns

ANGULAR ARCHITECTS

Intention, Store, Signals

View Models via Computed

Entities & Normalization

Custom Features, e.g. withDataService

State in Each Layer

Smart vs. Dumb